

Emulazione di un datacenter SDN multi-tenant

Matteo Amarante DE9000139 Antonio Ferrara DE9000166

1. Introduzione

Il progetto è stato sviluppato per emulare i meccanismi che stanno alla base di un datacenter gestito con tecnologie SDN. L'idea di fondo è quella di costruire un ambiente ridotto ma completo, nel quale la rete non sia configurata dispositivo per dispositivo, bensì governata da un controller esterno, e nel quale i servizi applicativi vengono installati in maniera automatica tramite dei meccanismi di auto scaling.

Il lavoro si è concentrato su tre proprietà:

- la **separazione fra piano di controllo e piano dati**, con gli switch ridotti a elementi di inoltro programmabili e ogni decisione presa dal controller attraverso il protocollo OpenFlow;
- l'**isolamento multi-tenant a livello 2** mediante 802.1Q, con l'idea che l'appartenenza al tenant sia decisa dalla rete e mai dichiarata dall'host;
- l'**elasticità del livello applicativo**, ottenuta affiancando a un load balancer una politica di scaling dei container.

L'infrastruttura di test è distribuita su due macchine virtuali: la prima ospita il controller Ryu, la seconda l'ambiente di emulazione ComNetsEmu, estensione di Mininet in cui gli host sono container Docker e i servizi possono essere avviati e spenti mentre la rete è in funzione.

2. Architettura del sistema

L'architettura complessiva del sistema è riportata in figura.

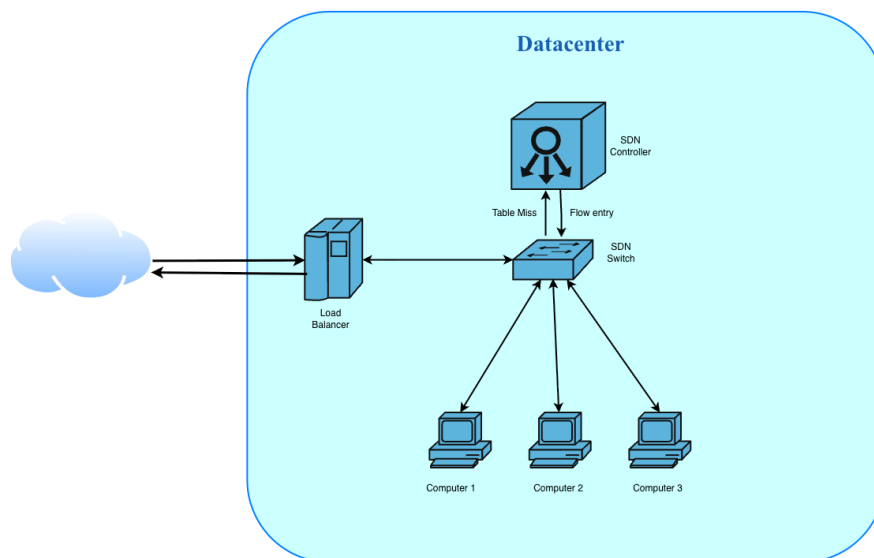


Figura 1: architettura del datacenter emulato

Il **load balancer** è l'unico componente esposto verso l'esterno: riceve le richieste HTTP e le smista, attraverso gli endpoint `/compute` e `/chat`, verso i container attivi.

Lo **switch SDN** costituisce l'intero piano dati della topologia. Non contiene alcuna configurazione statica di VLAN: le sue tabelle sono popolate esclusivamente dal controller. Quando un pacchetto non trova una regola corrispondente, lo switch lo inoltra al controller, meccanismo anche detto *reactive packet processing* (evento packet-in generato dalla regola di table-miss); il controller apprende la posizione dell'host, calcola le azioni e installa la relativa flow entry, così che i pacchetti successivi dello stesso flusso siano gestiti autonomamente dallo switch.

Il **controller Ryu** è eseguito su una macchina distinta e dialoga con lo switch unicamente in OpenFlow. I ruoli restano quindi separati: il controller decide come inoltrare i pacchetti, il load balancer a quale container mandare le richieste e quando crearne di nuovi.

I tre **computer** sono host docker su cui andremo a gestire i container dei servizi. Ciascuno di essi può ospitare fino a tre

container.

3. Il percorso di una richiesta

La figura segue la prima richiesta che entra nel sistema. Il load balancer la riceve dall'esterno e la inoltra a un container della calcolatrice, che risponde con il risultato. Il primo pacchetto del flusso non trova una regola nello switch e viene mandato al controller, che installa la flow entry corrispondente: da quel momento in poi lo switch inoltra da solo, senza più coinvolgere il controller.

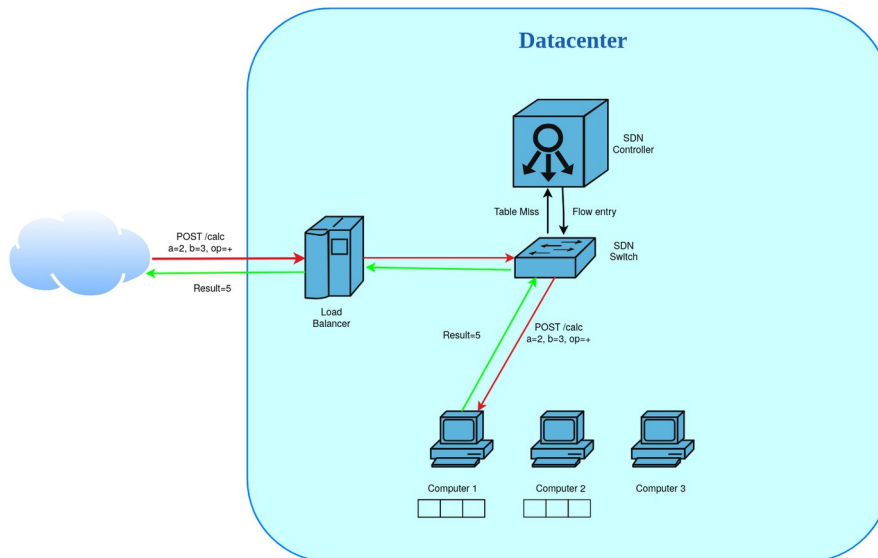


Figura 2: percorso della prima richiesta

I quadratini sotto ogni computer rappresentano gli slot di un container, tre in tutto. Nella figura il primo posto di computer1 risulta ancora occupato dalla richiesta precedente, e la nuova richiesta viene mandata al container su computer2: fra i container dello stesso servizio il load balancer sceglie infatti sempre il meno carico. Per la stessa ragione i container di un servizio vengono distribuiti su computer diversi prima di riempire un singolo host.

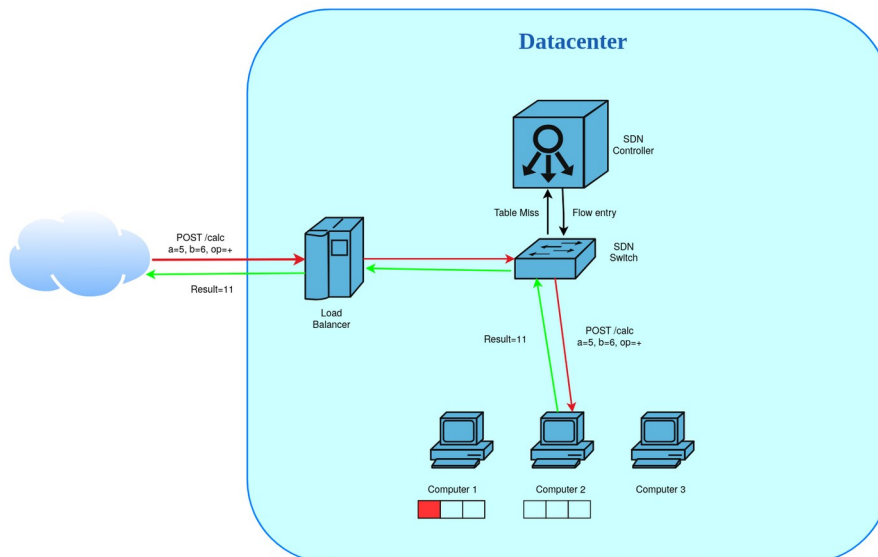


Figura 3: la richiesta successiva va sul computer meno carico

4. Isolamento multi-tenant

I tre computer sono divisi in due tenant: il tenant 10 raccoglie computer1 e computer2, sui quali gira il servizio di calcolo, mentre il tenant 20 è riservato a computer3, che ospita il chatbot. I due tenant condividono la stessa sottorete IP: l'isolamento è quindi realizzato interamente a livello 2 mediante le VLAN.

Il tenant non è dichiarato dall'host ma deciso dalla rete. Gli host non sono a conoscenza dell'esistenza delle VLAN e trasmettono traffico non taggato: è il controller ad associare ogni porta dello switch al tenant corrispondente. Se un host prova a mandare un frame già taggato, per entrare in un tenant che non è il suo, lo switch lo scarta. Sono infatti installate delle regole di guardia che valgono su tutte le porte tranne quella verso il load balancer, l'unica in cui il traffico taggato è legittimo, perché il load balancer è presente in entrambi i tenant esattamente come accade in un gateway reale. Il codice riporta le due parti di controller che realizzano questo comportamento: la mappa fra porte e tenant con le regole di guardia, e la funzione che stabilisce a quale tenant appartiene un frame in arrivo.

```
PORT_TENANT = {1: 10, 2: 10, 3: 20}      # porta di accesso -> tenant
TRUNK_PORTS = {ofproto_v1_3.OFPP_LOCAL}  # la porta verso il load balancer

def install_access_guards(self, datapath):
    for port in PORT_TENANT:
        if port in TRUNK_PORTS:
            continue
        match = parser.OFPMatch(
            in_port=port,
            vlan_vid=(ofproto.OFPPVID_PRESENT, ofproto.OFPPVID_PRESENT))
        self.add_flow(datapath, GUARD_PRIORITY, match, []) # [] = scarta

def tenant_of(self, in_port, vlan_hdr):
    if in_port in TRUNK_PORTS:
        return vlan_hdr.vid if vlan_hdr else None
    if vlan_hdr is not None:
        return None
    # frame taggato su porta di accesso
    return PORT_TENANT.get(in_port)
```

assegnazione del tenant e regole di guardia

Il traffico fra host di tenant diversi non viene scartato in prossimità della destinazione: non lascia neppure lo switch, come mostrato in figura. Il controller non installa alcuna regola che colleghi porte di tenant diversi e, quando la destinazione non è ancora nota, inoltra il pacchetto soltanto alle porte dello stesso tenant, mai in broadcast su tutta la rete.

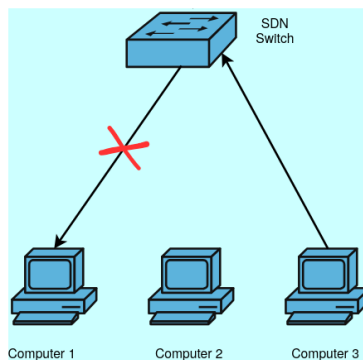


Figura 4: il traffico fra tenant diversi non attraversa lo switch

5. Load balancing e scaling

Ogni servizio ha il proprio insieme di container e ogni container accetta al più tre richieste contemporanee. Le richieste in corso vengono contate separatamente per ogni servizio, dal momento che un valore complessivo su servizi diversi non direbbe nulla di utile. La scelta del container, riportata nel codice, ricade sempre su quello con meno richieste in corso fra quelli del servizio.

```
def endpoint_libero(servizio):
    """Ritorna il container meno carico del servizio"""
    scelto = None
    for nome, endpoint in endpoints.items():
        if endpoint["servizio"] != servizio or endpoint["richieste"] >= MAX_PER_ENDPOINT:
            continue
        if scelto is None or endpoint["richieste"] < endpoints[chosen]["richieste"]:
            scelto = nome
    return scelto
```

scelta del container meno carico

Quando tutti i posti sono occupati la richiesta non viene rifiutata subito, ma resta in attesa per un massimo di 30 secondi, il tempo necessario perché lo scaling aggiunga capacità. Solo allo scadere dell'attesa il load balancer risponde

con un errore.

Un thread di monitoraggio osserva l'utilizzo di ogni servizio, cioè il rapporto fra richieste in corso e posti disponibili. Sopra l'80% viene creato un nuovo container, sotto il 30% per dieci secondi consecutivi ne viene rimosso uno. Le due soglie sono volutamente distanti e la rimozione avviene solo dopo un ritardo, in modo che il sistema non alterni di continuo creazioni e distruzioni a fronte di oscillazioni momentanee del traffico. Il codice mostra il ciclo che applica questa politica. Ogni giro si svolge in due fasi: prima vengono raccolti gli utilizzi di tutti i servizi, poi si decide servizio per servizio, così che le decisioni di uno stesso giro partano tutte dalla medesima fotografia del carico.

```
SCALE_UP_THRESHOLD = 0.80      # utilizzo oltre il quale si aggiunge un container
SCALE_DOWN_THRESHOLD = 0.30    # utilizzo sotto il quale se ne toglie uno
SCALE_DOWN_COOLDOWN = 10      # secondi consecutivi sotto soglia prima di togliere

def monitor_loop():
    sotto_soglia_da = {}
    for servizio in SERVIZI:
        sotto_soglia_da[servizio] = None

    while True:
        time.sleep(MONITOR_INTERVAL)

        letture = {}
        for servizio in SERVIZI:
            letture[servizio] = LoadBalancer.utilizzo(servizio)

        for servizio in SERVIZI:
            u = letture[servizio]
            attivi = quanti(servizio)

            if u >= SCALE_UP_THRESHOLD:
                sotto_soglia_da[servizio] = None
                if attivi < tetto(servizio):
                    LoadBalancer.aggiungi_backend(scale_up(servizio))

            elif u < SCALE_DOWN_THRESHOLD and attivi > MIN_CONTAINERS:
                if sotto_soglia_da[servizio] is None:
                    sotto_soglia_da[servizio] = time.time()
                elif time.time() - sotto_soglia_da[servizio] >= SCALE_DOWN_COOLDOWN:
                    nome = LoadBalancer.togli_backend(servizio)
                    if nome is not None:
                        scale_down(nome)
                        sotto_soglia_da[servizio] = None
            else:
                sotto_soglia_da[servizio] = None
```

la politica di scaling

Il container nuovo viene creato sul computer meno carico fra quelli assegnati al servizio: il servizio di calcolo può quindi arrivare a sei container, tre su computer1 e tre su computer2, mentre il chatbot si ferma a tre. In ogni caso resta sempre acceso almeno un container per servizio, così che nessuno resti privo di capacità.

6. Prove sperimentali

Le prove sono state condotte con uno script esterno che invia richieste al load balancer e ne legge periodicamente lo stato, riportando quanti container risultano attivi per ciascun servizio.

Il **primo test** verifica il funzionamento di base: una richiesta per servizio, entrambe servite correttamente, con un solo container acceso per parte.

```
vagrant@comnetsemu:~/ncis$ python3 test.py
== TEST 1 ==
POST http://localhost:8081/compute a=2 b=3 op=+
200 {"result":5}
POST http://localhost:8081/chat messaggio=ciao
200 {"risposta":"Ciao! Come stai?"}
container up   calc=1 chat=1
container up   calc=1 chat=1
```

Test 1: verifica funzionale dei due servizi

Il **secondo test** invia tre richieste contemporanee per servizio, tante quanti sono i posti di un container. L'utilizzo raggiunge la soglia superiore e il monitor crea un secondo container per ciascun servizio.

```

=== TEST 2 ===
POST http://localhost:8081/compute a=2 b=3 op=+
POST http://localhost:8081/chat messaggio=ciao
POST http://localhost:8081/compute a=2 b=3 op=+
POST http://localhost:8081/chat messaggio=ciao
POST http://localhost:8081/compute a=2 b=3 op=+
POST http://localhost:8081/chat messaggio=ciao
200 {"risposta":"Ciao! Come stai?"}
200 {"result":5}
200 {"result":5}
200 {"risposta":"Ciao! Come stai?"}
200 {"result":5}
200 {"risposta":"Ciao! Come stai?"}
container up   calc=1 chat=1
container up   calc=2 chat=2

```

Test 2: la saturazione dei posti fa creare un secondo container

Il **terzo test** è uno stress test, con un numero di richieste molto superiore alla capacità disponibile. I due insiemi crescono fino al proprio tetto, sei container per il calcolo e tre per il chatbot. Tutte le richieste sono comunque state servite: nessuna ha esaurito i 30 secondi di attesa.

```

200 {"risposta":"Ciao! Come stai?"}
200 {"risposta":"Ciao! Come stai?"}
200 {"risposta":"Ciao! Come stai?"}
200 {"risposta":"Ciao! Come stai?"}
container up   calc=6 chat=3
container up   calc=6 chat=3

```

Test 3: i due servizi raggiungono il tetto di sei e tre container

Il **quarto test** lascia il sistema inattivo. I container in eccesso vengono spenti uno alla volta, con il ritardo previsto fra una rimozione e la successiva, fino a tornare al minimo di un container per servizio.

```

=== TEST 4 ===
container up   calc=5 chat=3
container up   calc=5 chat=2
container up   calc=4 chat=2
container up   calc=4 chat=1

```

Test 4: con il sistema inattivo i container vengono spenti

7. Conclusioni

Il sistema realizzato mostra in un unico scenario i tre aspetti che ci eravamo proposti di indagare. La rete è governata interamente dal controller, che programma le tabelle dello switch a partire dai pacchetti che gli vengono inoltrati; l'isolamento fra tenant è ottenuto a livello 2 con le VLAN e resta valido anche con host che condividono la stessa sottorete IP; la capacità di servizio segue il carico, grazie a un load balancer che distribuisce le richieste e a una politica di scaling che crea e distrugge container mentre il sistema è in funzione.

Per migliorare la qualità dell'architettura, è possibile utilizzare un reverse proxy e una logica di load balancing associata ad ogni tenant. In questo modo, si può semplificare il meccanismo di scaling, garantendo la continuità del servizio ad ogni tenant e permettendogli di modificare la logica di scaling e load balancing per adattare i propri servizi alle funzionalità offerte dal datacenter. Inoltre, si possono aggiungere meccanismi di QoS in modo da garantire un servizio migliore a determinati clienti.